No testing or validation

Pitfall: Implementing policies that seem correct but haven’t been tested in real-world situations.

Avoid it: Validate policies against a schema, write test requests, and use automated analysis to ensure that they enforce the intended invariants—the conditions that must always hold true regardless of context. For example, an invariant might state that only project members can view project documents, or that only employees can share files externally.

At ACME, these lessons were learned the hard way: an early “all employees” permit unintentionally allowed interns to see financial forecasts. After discovering the issue during testing, the team adopted a stricter least-privilege approach and made schema validation a required step in policy review.

10.1.6 From principles to practice

Taken together, these principles show that creating effective policies involves more than just granting or denying access at a specific moment. Clear and consistent policies help prevent conflicts. Applying least privilege minimizes overexposure. Separating policy from data maintains the rules' resilience. Using granularity and grouping finds a balance between security and usability. Anchoring policies to durable entities, such as teams, ensures long-term stability. By applying these lessons, ACME’s identity team develops policies that are both secure and sustainable. In the next section, we will explore how these design principles are implemented when policies are managed as code, versioned, and tested alongside the software they protect.

10.2 Policy as code in practice

At ACME, the identity team reached a turning point when they realized that authorization rules were spread across applications in code comments, configuration files, and even ad hoc database queries. Each product team handled access differently. The Customer Collaboration platform checked